

Git Command Master Reference

A comprehensive compilation of 65 essential Git commands, descriptions, and real-world examples.

1. SETUP & INITIALIZATION

COMMAND	DESCRIPTION	PRACTICAL EXAMPLE
<code>git init</code>	Create a new local Git repository.	<code>git init my-repo</code>
<code>git clone</code>	Clone a repository into a new directory.	<code>git clone https://github.com/user/repo.git</code>
<code>git config</code>	Get and set repository or global options.	<code>git config --global user.email "you@example.com"</code>
<code>git config --list</code>	List all active Git configuration settings from system, global, and local layers.	<code>git config --list</code>

2. BASIC SNAPSHOTTING & STAGING

COMMAND	DESCRIPTION	PRACTICAL EXAMPLE
<code>git add</code>	Stage changes for the next commit.	<code>git add file.txt</code>
<code>git commit</code>	Record staged changes in the repository.	<code>git commit -m "Add feature"</code>
<code>git commit --amend</code>	Modify the most recent commit by combining staged changes or altering the message.	<code>git commit --amend -m "Updated fix"</code>
<code>git status</code>	Show the working tree status.	<code>git status</code>
<code>git rm</code>	Remove files from working tree and index.	<code>git rm obsolete.txt</code>
<code>git mv</code>	Move or rename a file, directory, or symlink.	<code>git mv oldname newname</code>
<code>git restore</code>	Restore working tree files or discard local changes (modern form).	<code>git restore src/app.js</code>
<code>git clean</code>	Remove untracked files from the working tree.	<code>git clean -fd</code>

3. BRANCHING & MERGING

COMMAND	DESCRIPTION	PRACTICAL EXAMPLE
<code>git branch</code>	List, create, or delete branches.	<code>git branch feature/login</code>
<code>git branch -d</code>	Delete a local branch that has been safely merged downstream.	<code>git branch -d feature/old-login</code>
<code>git branch -m</code>	Rename the current active branch or a specified branch.	<code>git branch -m feature/new-login</code>
<code>git checkout</code>	Switch branches or restore files (older legacy form).	<code>git checkout feature/login</code>
<code>git checkout -b</code>	Create a new branch and immediately switch to it (legacy shorthand).	<code>git checkout -b feature/payment</code>
<code>git switch</code>	Switch branches (safer, dedicated modern command).	<code>git switch -c feature/login</code>
<code>git merge</code>	Join two or more development histories together.	<code>git merge main</code>
<code>git merge --abort</code>	Stop the merge process and reconstruct the pre-merge state if conflicts occur.	<code>git merge --abort</code>
<code>git rebase</code>	Reapply commits on top of another base tip.	<code>git rebase main</code>
<code>git rebase -i</code>	Interactive rebase to edit, squash, or reorder commits.	<code>git rebase -i HEAD-5</code>
<code>git worktree</code>	Manage multiple working trees attached to the same repository.	<code>git worktree add ../repo-worktree feature/login</code>

4. INSPECTING HISTORY & COMPARISON

COMMAND	DESCRIPTION	PRACTICAL EXAMPLE
<code>git status</code>	Show the working tree status.	<code>git status</code>
<code>git log</code>	Show commit history.	<code>git log --oneline --graph</code>
<code>git shortlog</code>	Summarize git log output by author.	<code>git shortlog -sne</code>
<code>git reflog</code>	Show a log of where HEAD and branches have been over time.	<code>git reflog</code>
<code>git diff</code>	Show changes between commits, branches, or working tree.	<code>git diff HEAD-1 HEAD</code>
<code>git diff --staged</code>	Show changes between files staged for the next commit and the last commit.	<code>git diff --staged</code>
<code>git show</code>	Show various types of objects (commits, tags, files).	<code>git show HEAD-1</code>
<code>git blame</code>	Show what revision and author last modified each line of a file.	<code>git blame src/app.js</code>
<code>git grep</code>	Search for regular expression patterns in the repository content.	<code>git grep "TODO"</code>
<code>git describe</code>	Describe a commit using the nearest readable tag.	<code>git describe --tags --always</code>

5. SHARE & SYNC REPOSITORIES (REMOTES)

COMMAND	DESCRIPTION	PRACTICAL EXAMPLE
<code>git remote</code>	Manage the set of tracked remote repositories.	<code>git remote -v</code>
<code>git remote add</code>	Associate a local repository with a new remote URL connection.	<code>git remote add origin https://github.com/user/repo.git</code>

COMMAND	DESCRIPTION	PRACTICAL EXAMPLE
<code>git remote set-url</code>	Change the target URL of an existing remote repository connection.	<code>git remote set-url origin https://github.com/user/new.git</code>
<code>git fetch</code>	Download objects and refs from another repository.	<code>git fetch origin</code>
<code>git fetch --all</code>	Fetch updates from all tracked remotes simultaneously.	<code>git fetch --all --prune</code>
<code>git pull</code>	Fetch from and integrate with another repository or a local branch.	<code>git pull origin main</code>
<code>git push</code>	Update remote refs along with associated objects.	<code>git push origin feature/login</code>
<code>git push --tags</code>	Push all local release tags up to the remote server repository.	<code>git push origin --tags</code>
<code>git push origin -d</code>	Delete a specified branch or tag from the remote repository.	<code>git push origin --delete feature/login</code>

6. STASHING TEMPORARY CHANGES

COMMAND	DESCRIPTION	PRACTICAL EXAMPLE
<code>git stash</code>	Stash local modifications away to quickly clean the working directory.	<code>git stash push -m "WIP: refactor"</code>

COMMAND	DESCRIPTION	PRACTICAL EXAMPLE
<code>git stash pop</code>	Apply the latest stashed state and remove it from the stash list.	<code>git stash pop</code>
<code>git stash apply</code>	Apply a specific stash state without removing it from the list.	<code>git stash apply stash@{1}</code>
<code>git stash list</code>	List all currently saved stashed changesets.	<code>git stash list</code>
<code>git stash drop</code>	Permanently discard a specific stashed changeset from your list.	<code>git stash drop stash@{0}</code>
<code>git stash clear</code>	Completely wipe out and remove all stashed records.	<code>git stash clear</code>

7. ADVANCED UNDO, RESCUE & PATCHING

COMMAND	DESCRIPTION	PRACTICAL EXAMPLE
<code>git cherry-pick</code>	Apply changes introduced by existing individual historical commits.	<code>git cherry-pick a1b2c3d</code>
<code>git cherry</code>	List commits that have not yet been merged upstream.	<code>git cherry -v origin/main</code>
<code>git revert</code>	Create a new safe commit that completely undoes mutations from a prior commit.	<code>git revert a1b2c3d</code>
<code>git reset</code>	Reset current HEAD pointer to a specified state (soft/mixed).	<code>git reset --soft HEAD-1</code>

COMMAND	DESCRIPTION	PRACTICAL EXAMPLE
git reset --hard	Reset current branch HEAD, wiping out all uncommitted and tracked modifications.	<code>git reset --hard HEAD~1</code>
git bisect	Use binary search strategy to locate the commit that introduced a regression/bug.	<code>git bisect start; git bisect bad; git bisect good <commit></code>
git format-patch	Prepare electronic emailable patches directly out of historical commits.	<code>git format-patch -3</code>
git am	Apply individual patches from structured mailbox (mbox) formatted files.	<code>git am 0001-fix-bug.patch</code>
git apply	Apply a raw unified diff patch file onto files or index directly.	<code>git apply fix.patch</code>

8. MAINTENANCE, ADMINISTRATION & SUBMODULES

COMMAND	DESCRIPTION	PRACTICAL EXAMPLE
git tag	Create, list, or delete release tags mapping specific history checkpoints.	<code>git tag -a v1.0 -m "Release 1.0"</code>

COMMAND	DESCRIPTION	PRACTICAL EXAMPLE
git submodule	Manage complex nested embedded repositories within a parent repository.	<code>git submodule add https://github.com/lib/lib.git deps/lib</code>
git submodule update	Initialize, fetch, and checkout all configured submodules recursively.	<code>git submodule update --init --recursive</code>
git archive	Create a clean zip or tar archive of elements originating from a named tree.	<code>git archive --format tar --output=project.tar HEAD</code>
git fsck	Verify object database health, matching connectivity and validity.	<code>git fsck --full</code>
git gc	Cleanup unnecessary garbage tracking files and optimize database performance.	<code>git gc --prune=now</code>
git bundle	Pack or unpack binary multi-object offline transport container bundles.	<code>git bundle create repo.bundle --all</code>
git notes	Add or inspect textual meta-notes attached to active objects safely.	<code>git notes add -m "Reviewed"</code>

COMMAND	DESCRIPTION	PRACTICAL EXAMPLE
<code>git rerere</code>	Reuse recorded automated resolutions of identical long-term merge conflicts.	<code>git rerere enabled</code>